

PCAP LAB 8

Name: Abhinav Singh Bhagtana

Roll no: A-029

Reg no: 230905225

Q1.

1. Write a program in CUDA to add two Matrices for the following specifications:
 - a. Each row of resultant matrix to be computed by one thread.
 - b. Each column of resultant matrix to be computed by one thread.
 - c. Each element of resultant matrix to be computed by one thread.

```
#include "cuda_runtime.h" #include <stdio.h>
```

```
//solving this using a 1D block and 1D grid global void OneThreadPerRow(int A, int B, int C, int m, int n) { int idx = blockDim.x * blockIdx.x + threadIdx.x; //each index corresponds to a row if (idx < m) { //c is the column for(int c = 0; c < n; c++) { C[idxn + c] = A[idxn + c] + B[idxn + c]; } } } //solving this using a 1D block and 1D grid global void OneThreadPerColumn(int *A, int *B, int *C, int m, int n) { int idx = blockDim.x * blockIdx.x + threadIdx.x; //each index corresponds to a column if (idx < n) { //k is the row number for(int k = 0; k < m; k++) { C[idx + k * n] = A[idx + k * n] + B[idx + k * n]; } } } //solving using 2D block and 1D grid global void OneThreadPerElement(int *A, int *B, int C, int m, int n) { int idx = blockIdx.x * (blockDim.x * blockDim.y) + threadIdx.y * blockDim.x + threadIdx.x; if(idx < mn) C[idx] = A[idx] + B[idx]; }
```

```
int main() { int m,n; printf("enter dimensions of matrices: "); scanf("%d%d",&m,&n);
```

```
int A[m][n], B[m][n], C[m][n];
```

```
int *d_A, *d_B, *d_C;
```

```
int size = sizeof(int)*m*n;
```

```
printf("enter values for matrix A:\n");
```

```
for(int i = 0; i < m; i++)
```

```
    for(int j = 0; j < n; j++)
```

```
        scanf("%d",&A[i][j]);
```

```
printf("enter values for matrix B:\n");
```

```
for(int i = 0; i < m; i++)
```

```
    for(int j = 0; j < n; j++)
```

```
        scanf("%d",&B[i][j]);
```

```

//tell the gpu to allocate space
cudaMalloc((void**)&d_A,size);
cudaMalloc((void**)&d_B,size);
cudaMalloc((void**)&d_C,size);

//copy A and B to gpu memory
cudaMemcpy(d_A,A,size,cudaMemcpyHostToDevice);
cudaMemcpy(d_B,B,size,cudaMemcpyHostToDevice);

//call Kernels
printf("Result computed using one thread per row\n");
dim3 block1(256);
dim3 grid1((m * n + 255) / 256);
OneThreadPerRow<<<grid1,block1>>>(d_A,d_B,d_C,m,n);
cudaMemcpy(C,d_C,size,cudaMemcpyDeviceToHost);
for(int i =0;i < m;i++){
    for(int j=0;j<n;j++){
        printf("%d\t",C[i][j]);
        printf("\n");
    }

    printf("Result computed using one thread per row\n");
    dim3 grid2((n + 255) / 256);
    OneThreadPerColumn<<<grid2, block1>>>(d_A, d_B, d_C, m, n);
    cudaMemcpy(C,d_C,size,cudaMemcpyDeviceToHost);
    for(int i =0;i < m;i++){
        for(int j=0;j<n;j++){
            printf("%d\t",C[i][j]);
            printf("\n");
        }

        printf("Result computed using one thread per element\n");
        dim3 block(m,n);
        dim3 grid(1);
        OneThreadPerElement<<<grid,block>>>(d_A,d_B,d_C,m,n);
        cudaMemcpy(C,d_C,size,cudaMemcpyDeviceToHost);
        for(int i =0;i < m;i++){
            for(int j=0;j<n;j++){
                printf("%d\t",C[i][j]);
                printf("\n");
            }
        }
    }
}

```

```

}

cudaFree(d_A);
cudaFree(d_B);
cudaFree(d_C);
return 0;

}

```

OUTPUT:

```

STUDENT@MIT-ICT-LAB5-29:~/230905225/week8$ nvcc q1.cu
STUDENT@MIT-ICT-LAB5-29:~/230905225/week8$ ./a.out
enter dimensions of matrices: 3 2
enter values for matrix A:
1 2 3 4 5 6
enter values for matrix B:
3 2 1 4 5 6
Result computed using one thread per row
4      4
4      8
10     12
Result computed using one thread per row
4      4
4      8
10     12
Result computed using one thread per element
4      4
4      8
10     12
STUDENT@MIT-ICT-LAB5-29:~/230905225/week8$ 

```

Q2.

2. Write a program in CUDA to multiply two Matrices for the following specifications:
 - a. Each row of resultant matrix to be computed by one thread.
 - b. Each column of resultant matrix to be computed by one thread.
 - c. Each element of resultant matrix to be computed by one thread.

```
#include "cuda_runtime.h" #include <stdio.h>
```

```
//solved 1D grid 1D block global void OneThreadPerRow(int *A, int *B, int *C, int m, int
n, int k) { int row = blockDim.x * blockIdx.x + threadIdx.x;
```

```

if (row < m)
{

```

```

for (int j = 0; j < k; j++) // for each column of C
{
    int sum = 0;
    for (int t = 0; t < n; t++) // dot product
    {
        sum += A[row * n + t] * B[t * k + j];
    }
    C[row * k + j] = sum;
}
}

```

```

}

```

//solved using 1D grid 1D block **global** void OneThreadPerColumn(int *A, int *B, int *C, int m, int n, int k) { int col = blockDim.x * blockIdx.x + threadIdx.x;

```

if (col < k)
{
    for (int i = 0; i < m; i++) // for each row
    {
        int sum = 0;
        for (int t = 0; t < n; t++) // dot product
        {
            sum += A[i * n + t] * B[t * k + col];
        }
        C[i * k + col] = sum;
    }
}
}

```

} // solved using 1D grid 1D block **global** void OneThreadPerElement(int *A, int *B, int *C, int m, int n, int k) { // Total number of elements in C is m * k int idx = blockDim.x * blockIdx.x + threadIdx.x;

```

if (idx < m * k)
{
    // Map 1D index to 2D row and column
    int row = idx / k;
    int col = idx % k;

    int sum = 0;

```

```

    for (int t = 0; t < n; t++) // Standard dot product loop
    {
        sum += A[row * n + t] * B[t * k + col];
    }
    C[idx] = sum;
}

}

// Helper function to print matrix void printMatrix(int* mat, int rows, int cols, const char*
label) { printf("\nMatrix %s (%dx%d):\n", label, rows, cols); for (int i = 0; i < rows; i++) { for
(int j = 0; j < cols; j++) { printf("%d\t", mat[i * cols + j]); } printf("\n"); } }

int main() { int m, n, k; printf("Enter dimensions (m, n, k): "); if (scanf("%d %d %d", &m,
&n, &k) != 3) return -1;

size_t sizeA = m * n * sizeof(int);
size_t sizeB = n * k * sizeof(int);
size_t sizeC = m * k * sizeof(int);

int *h_A = (int*)malloc(sizeA);
int *h_B = (int*)malloc(sizeB);
int *h_C = (int*)malloc(sizeC);

printf("Enter matrix A (%dx%d):\n", m, n);
for (int i = 0; i < m * n; i++) scanf("%d", &h_A[i]);
printf("Enter matrix B (%dx%d):\n", n, k);
for (int i = 0; i < n * k; i++) scanf("%d", &h_B[i]);

int *d_A, *d_B, *d_C;
cudaMalloc(&d_A, sizeA);
cudaMalloc(&d_B, sizeB);
cudaMalloc(&d_C, sizeC);

cudaMemcpy(d_A, h_A, sizeA, cudaMemcpyHostToDevice);
cudaMemcpy(d_B, h_B, sizeB, cudaMemcpyHostToDevice);

int threadsPerBlock = 256;

// --- Launch 1: One Thread Per Row ---
int blocksPerRow = (m + threadsPerBlock - 1) / threadsPerBlock;

```

```

OneThreadPerRow<<<blocksPerRow, threadsPerBlock>>>(d_A, d_B, d_C, m, n, k);

// Copy and Print
cudaMemcpy(h_C, d_C, sizeC, cudaMemcpyDeviceToHost);
printMatrix(h_C, m, k, "Result (OneThreadPerRow)");

// --- Launch 2: One Thread Per Column ---
int blocksPerCol = (k + threadsPerBlock - 1) / threadsPerBlock;
OneThreadPerColumn<<<blocksPerCol, threadsPerBlock>>>(d_A, d_B, d_C, m, n, k);

// Copy and Print
cudaMemcpy(h_C, d_C, sizeC, cudaMemcpyDeviceToHost);
printMatrix(h_C, m, k, "Result (OneThreadPerColumn)");

// --- Launch 3: One Thread Per Element ---
int blocksPerElem = (m * k + threadsPerBlock - 1) / threadsPerBlock;
OneThreadPerElement<<<blocksPerElem, threadsPerBlock>>>(d_A, d_B, d_C, m, n, k);

// Copy and Print
cudaMemcpy(h_C, d_C, sizeC, cudaMemcpyDeviceToHost);
printMatrix(h_C, m, k, "Result (OneThreadPerElement)");

// Cleanup
cudaFree(d_A); cudaFree(d_B); cudaFree(d_C);
free(h_A); free(h_B); free(h_C);

return 0;

}

```

OUTPUT:

```
STUDENT@MIT-ICT-LAB5-29:~/230905225/week8$ nvcc q2.cu
STUDENT@MIT-ICT-LAB5-29:~/230905225/week8$ ./a.out
Enter dimensions (m, n, k): 2 3 4
Enter matrix A (2x3):
1 2 3 4 5 6
Enter matrix B (3x4):
1 2 3 4 5 6 7 8 9 10 11 12

Matrix Result (OneThreadPerRow) (2x4):
38      44      50      56
83      98      113     128

Matrix Result (OneThreadPerColumn) (2x4):
38      44      50      56
83      98      113     128

Matrix Result (OneThreadPerElement) (2x4):
38      44      50      56
83      98      113     128
STUDENT@MIT-ICT-LAB5-29:~/230905225/week8$
```